

Applying the Scalable Tensor Factorization to the Twitter Hybrid Network

16 December 2013

peratham@umiacs.umd.edu

1 Introduction

Twitter is an instance of microblog which has various interactions between users. Figure 1. describes the interactions between Twitter users by denoting users as nodes and interactions as edges. There are three types of interactions in Twitter which are following relation, mention relation and retweet relation. The network is a map of influences which flows from nodes to nodes via connected edges. In this Twitter network, we consider two problems of identifying top K users who will retweet or mention the focal user in the future.

Predicting the future user interactions based on the influence flows in the network can be viewed as the link prediction problem. Link prediction has two scenarios. The first scenario considers the static snapshot and the objective is to recover the missing links. The second scenario is to consider an evolving network in time and predict the future snapshot or multiple future snapshots. This project considers the second scenario for one future snapshot.

Traditional link prediction considers a homogeneous network where there is only one type of interaction between users. However, many real world networks such as the Twitter network are heterogeneous. There can be many type of users or interactions. Tensors provide a natural way to represent heterogeneous data such as social network data in multidimensional structure. By factorizing the tensor, the latent factors can be recovered and then used for predicting missing links. For this Twitter network, the future retweet and mention prediction can be solved simultaneously.

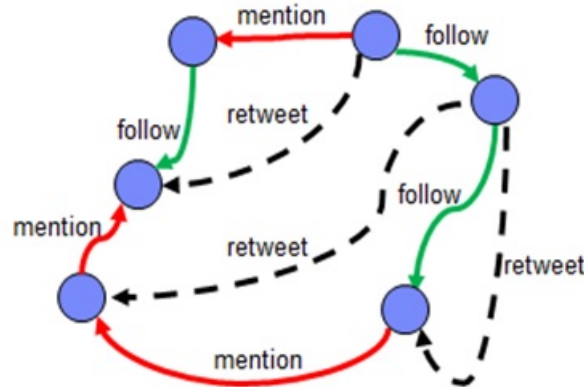


Figure 1: The Twitter hybrid network. [5]

2 CANDECOMP/PARAFACS (CP) and CP-WOPT Algorithms

Since the n -mode tensor is the product of n vector spaces with each owned coordinate system [4], CANDECOMP/PARAFACS (CP) decomposition finds the factors that form those vector spaces. Consider a three-mode tensor $X \in \mathbb{R}^{m \times n \times t}$, CP finds the decomposition as a summation of rank-one tensors

$$X \approx Z = \sum_i \lambda_i a_i \circ b_i \circ c_i, \quad (1)$$

where $\circ$ denotes the outer product, latent factors $a_i \in \mathbb{R}^m$, $b_i \in \mathbb{R}^n$ and $c_i \in \mathbb{R}^t$ are grouped by $\lambda_i \in \mathbb{R}^+$. Each subtensor $(\lambda_i a_i \circ b_i \circ c_i)$ is a component of X . Similar to SVD which decomposes the matrix to the sum of rank-one matrices, CP can be viewed as finding a spectral sum for a general n dimensional tensor. However, unlike SVD, latent factors of CP are not orthogonal.

For link prediction, the entries in the reconstructed tensor Z with the predefined rank R can be used as the potential scores to rank the potential users.

By the fact that computing the tensor factorization may not scale well, Acar et al.[1] proposed a first-order optimization approach CP-WOPT to solve the weighted least square problem which models just the known entries for the missing data scenario. This fits into the link prediction problem since social network data is very sparse and contain a lot of missing values. so modeling zero entries like CP may be inappropriate. CP-WOPT is well suited to the problem since it can factorize the tensor with noises and up to 99% missing data.

CP minimizes the following function,

$$X \approx Z = f(a, b, c) = \frac{1}{2} \sum_i \sum_j \sum_k (z_{ijk} - \sum_r a_{ir} b_{jr} c_{kr})^2. \quad (2)$$

CP-WOPT tries to model only the known entries and ignores missing entries completely by weighted optimization.

$$Z = f(a, b, c) = \frac{1}{2} \sum_i \sum_j \sum_k \{w_{ijk} (z_{ijk} - \sum_r a_{ir} b_{jr} c_{kr})\}^2. \quad (3)$$

$$w_{ijk} = \begin{cases} 1 & \text{if } x_{ijk} \text{ is known} \\ 0 & \text{if } x_{ijk} \text{ is missing} \end{cases}$$

The gradient with respect to a dimension a is as follows,

$$\frac{\partial f(A, B, C)}{\partial A} = (W * X - Z)(B \circ C), \quad (4)$$

where A, B, C denote the matrices which contain the latent factors. With the objective function and gradient function for each parameter, any first order optimization methods can be employed.

3 Experimental Results

This project consists of comparing CP and CP-WOPT by applying to Twitter data in a context of link prediction [5]. All experiments are performed in MATLAB on a laptop with Intel Core i7 4700MQ CPU and 12 GB RAM. Timings were performed using matlab ‘tic’ and ‘toc’ functions.

The implementation involves using the tensor toolbox [2] for representing sparse tensors since MATLAB only provides a sparse matrix representation. The tensor toolbox provides a way to represent up to N dimensional dense and sparse tensor representation. For the optimization solver, the Poblano toolbox [3] is used. Even though CP-WOPT is already implemented in the tensor toolbox, interfacing between the tensor representation offered by the tensor toolbox and the standard optimization solver is worth investigating. So, this is the reimplementation to understand the original code.

3.1 Data

The data is from [5] which was from using the Twitter API to construct the network. Starting from a seed user, the collection expanded via his follower network until 15,000 users are reached. The collection had been conducted for two months from April 25, 2011 to June 25, 2011. The first month was used as the training set and the evaluation was done on the second month.

At first, the tensor factorizations were performed on the full 15,000 users dataset, however, memory overflows were observed when trying to reconstruct the tensor Z . Hence, I sampled the data by considering only 1,000 users which is a much more manageable size.

Time range	04/25/2011–06/25/2011
Number of tweets	10,979,278
Number of retweets by and of the 15K active users	147,970
Number of mentions by and of the 15K active users	584,597
Avg_#_indegree of the follower network within the 15K nodes	219.59

Figure 2: Dataset description. [5]

3.2 Evaluation

Like in the original paper [5], Mean Average Precision (MAP) is used for evaluation. MAP is a widely measure to evaluate ranking methods. For each focal user, Average Precision (AP@K) is computed by thresholding the precision measure P at the cutoff K which is the top K users to be identified then divided

Method	Mention Prediction MAP	Retweet Prediction MAP
CP	0.2785	0.2396
CP-WOPT (ncg)	0.0933	0.0649
CP-WOPT (l-bfgs)	0.0755	0.0535

Table 1: The performance of CP and CP-WOPT on the hybrid network.

Method	Hybrid Network (sec)	unweighted Hybrid Network (sec)	Mention - Retweet (sec)
CP	684.95	159.74	12.59
CP-WOPT (ncg)	16022.96	3354.86	947.48
CP-WOPT (l-bfgs)	11575.43	3136.31	572.47

Table 2: The runtime of CP and CP-WOPT on the hybrid network.

by its position in the ranked list m . For this experiment, K is set to 20.

$$AP@K = \sum_{k=1}^K P(k)/\min(m, K). \quad (5)$$

Then, MAP can be computed by averaging AP@K across all focal users.

$$MAP@K = \sum_{i=1}^N AP@K/N. \quad (6)$$

3.3 Experiments on the Hybrid Network

The hybrid network was constructed by stacking the adjacency matrices of the mention network, the retweet network and the weighted following network as in [5]. Then, CP and CP-WOPT are used for decomposition. CP in the tensor toolbox is used. CP-WOPT is optimized by nonlinear conjugate gradient (ncg) and limited-memory bfgs (l-bfgs) by using the Poblano toolbox.

CP has a better runtime than CP-WOPT and has better MAP scores. The reason for the degraded accuracy of CP-WOPT may be the weighted following network is not sparse so the experiment was conducted again with the unweighted following network. With the unweighted following network in the hybrid network, CP performance reduced slightly. CP-WOPT performance improves but still not comparable to CP. So, the following network was excluded with the hope that some noises will be reduced. Without the following network in the hybrid network, CP performance again reduced slightly. CP-WOPT performance improves but still not comparable to CP. CP-WOPT with ncg has the better performance than l-bfgs but with longer runtime. However, ncg takes less number of iterations compared to l-bfgs (644 iterations and 1,000 iterations) but each iteration of ncg is much longer.

Method	Mention Prediction MAP	Retweet Prediction MAP
CP	0.2616	0.2389
CP-WOPT (ncg)	0.2104	0.1825
CP-WOPT (l-bfgs)	0.2147	0.1943

Table 3: The performance of CP and CP-WOPT on the hybrid network with unweighted following network.

Method	Mention Prediction MAP	Retweet Prediction MAP
CP	0.2520	0.2228
CP-WOPT (ncg)	0.2159	0.1905
CP-WOPT (l-bfgs)	0.1970	0.1837

Table 4: The performance of CP and CP-WOPT on the mention-retweet network.

The optimization parameters are varied for CP-WOPT so that the solver can take advantages of the tensor properties. The results indicate that by defining the dense solver to ncg works the best. For l-bfgs, the sparse option works the best.

4 Conclusions and Future Works

This project is about implementing and applying CP-WOPT for Twitter data and observe the scalability problem compared to the traditional CP implementation by Alternating Least Square (ALS). First, CP-WOPT is more scalable than CP in term of memory complexity but its time complexity depends on the optimization solver. The final tensor reconstruction for prediction is the bottleneck step that should not be dense. This should be addressed in the future work. Second, the experimental results show that by focusing on the known entries does not help future prediction since the MAP does not improve especially for future mention prediction. Third, the time for optimization is also longer than original CP method. Ncg takes less

Method	Mention Prediction MAP	Retweet Prediction MAP
CP-WOPT (ncg - limited sparse)	0.2159	0.1905
CP-WOPT (ncg - sparse)	0.2152	0.2198
CP-WOPT (ncg - dense)	0.2210	0.2190
CP-WOPT (l-bfgs - limited sparse)	0.1970	0.1837
CP-WOPT (l-bfgs - sparse)	0.2125	0.1879
CP-WOPT (l-bfgs - dense)	0.2103	0.1783

Table 5: The performance of CP-WOPT on the hybrid network excluding the following network when the optimization parameters and tensor types are varied.

iterations but more time per iteration than l-bfgs. Fourth, noisy features for CP-WOPT can degrade a lot of performance. This can be observed from the performance gain when removing the following network.

For future work, more optimization solvers may be interesting to investigate. Moreover, the weighting tensor ‘W’ when considers some unknown entries may help the performance. In addition, determining the optimal rank ‘R’ is still an open question.

References

- [1] Evrim Acar, Daniel M Dunlavy, Tamara G Kolda, and Morten Mørup. Scalable tensor factorizations for incomplete data. *Chemometrics and Intelligent Laboratory Systems*, 106(1):41–56, 2011.
- [2] Brett W. Bader, Tamara G. Kolda, et al. Matlab tensor toolbox version 2.5. Available online, January 2012.
- [3] Daniel M. Dunlavy, Tamara G. Kolda, and Evrim Acar. Poblano v1.0: A matlab toolbox for gradient-based optimization. Technical Report SAND2010-1422, Sandia National Laboratories, Albuquerque, NM and Livermore, CA, March 2010.
- [4] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.
- [5] Shanchan Wu and Louiqa Raschid. Prediction in a microblog hybrid network. In *Seventh ACM International Conference on Web Search and Data Mining (WSDM)*, 2014.

Matlab Code

```

function [ Px ] = my_cp_wopt( P, R , t_option )
%MY_CP_WOPT Summary of this function goes here
% Detailed explanation goes here
% INPUT : P is sptensor
%          R is the tensor rank
%          t_option in {'auto','sparse','sparse_lowmem'}
% OUTPUT : Px is the kruskal tensor contains factors
W = P > 0;

%% make function handle
opthandle = @ncg;
% opthandle = @lbfgs;
normPsqr = norm(P)^2;
options = feval(opthandle , 'defaults ');
fflag = ~isequal(t_option , 'sparse_lowmem ');
funhandle = @(x) my_cp_wopt_fun(P,W,x,normPsqr,t_option , fflag );

%% Random Initial Guess
sz = size(P);
N = length(sz);
P0 = cell(N,1);
for n=1:N
    P0{n} = randn(sz(n),R);
    for j=1:R
        P0{n}(:,j) = P0{n}(:,j) / norm(P0{n}(:,j)); % normalize
    end
end

%% Call Solver
out = feval(opthandle , funhandle , tt_fac_to_vec(P0), options);
P = ktensor(tt_cp_vec_to_fac(out.X,Z));

%% Clean up final result
% Arrange the final tensor so that the columns are normalized.
P = arrange(P);
% Fix the signs
Px = fixsigns(P);
end

function [f,g] = my_cp_wopt_fun(P,W,x,normPsqr,t_option , fflag)

```

```

A = tt_cp_vec_to_fac(x,W);

if isequal(t_option,'sparse')
    Pvals = tt_cp_wfg_sparse_setup(P,W); % filter the sub vals by W
    N = ndims(W);
    R = size(A{1},2);
    sz = cellfun(@(x) size(x,1),A);
    Wsubs = W.subs;
    Wvals = W.vals;
    Nvals = length(Wvals);
    %% Compute B = W.*ktensor(A)
    Bvals = zeros(Nvals,1);
    for r = 1:R
        newvals = Wvals;
        for n = 1:N
            bigArn = A{n}(Wsubs(:,n),r);
            newvals = newvals .* bigArn;
        end
        Bvals = Bvals + newvals;
    end
    %% function value: f = 0.5 * normPsqr - innerprod(Z,B) + 0.5 * norm(B)^2
    f = 0.5 * normPsqr - Pvals'*Bvals + 0.5 * sum(Bvals.^2);

    %% gradient computation
    Tvals = Pvals - Bvals;

    G = cell(N,1);
    for n = 1:N
        G{n} = zeros(size(A{n}));
    end

    for r = 1:R
        if (fflag)
            bigAr = cell(N,1);
            for n = 1:N
                bigAr{n} = A{n}(Wsubs(:,n),r);
            end
            for SkipN = 1:N
                newvals = Tvals;
                for n = [1:SkipN-1,SkipN+1:N]
                    newvals = newvals .* bigAr{n};
                end
            end
        end
    end
end

```



```

        G{SkipN}(:, r) = accumarray(Wsubs(:, SkipN), newvals, [sz(SkipN) 1])
    end
else
    for SkipN = 1:N
        newvals = Tvals;
        for n = [1:SkipN-1, SkipN+1:N]
            bigArn = A{n}(Wsubs(:, n), r);
            newvals = newvals .* bigArn;
        end
        G{SkipN}(:, r) = accumarray(Wsubs(:, SkipN), newvals, [sz(SkipN) 1])
    end
end

end

for n = 1:N
    G{n} = -G{n};
end

else if isequal(t_option, 'auto')
    B = W.*ktensor(A);
    % function value
    f = 0.5 * normPsqr - innerprod(P, B) + 0.5 * norm(B)^2;

    % gradient computation
    N = ndims(P);
    G = cell(N, 1);
    T = P - B;
    for n = 1:N
        G{n} = zeros(size(A{n}));
        G{n} = -mttkrp(T, A, n);
    end
end

end

g = tt_fac_to_vec(G);

```